

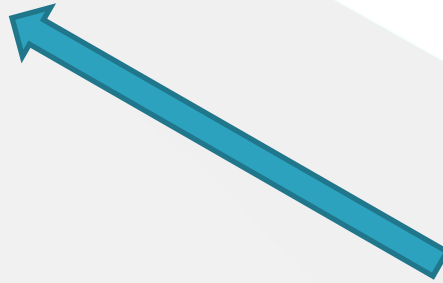
Functions in C

CSE 1112

Lecturer Raiyan Rahman

Department of CSE, UIU

Structure of a Book!



How a book is structured

Think about your textbook - teach yourself C (or any book!) and how it's structured.

1. They have (very) large texts, usually divided into chapters.
2. A table of content is given in the front to find what you want to study easily from this very large (say 772 pages) textbook!
3. If any chapter needs detailed info they often refer to other chapters. For example, while discussing strings, they may say:

Strings are a special type of arrays (to refresh your knowledge on arrays, see chapter 5, page 129)

Why?

Why do you think structuring like this helps?

1. Easier for the reader to find a specific topic, rather than getting lost inside the 772 page book.
2. Easier for the writer to write as well.
3. Can refer to other book chapters when needed. Saves time and makes it efficient.
4. Also, writers can collaborate, say they write each chapter that they have expertise on.

Codes are not that different!

Codes are not that different!

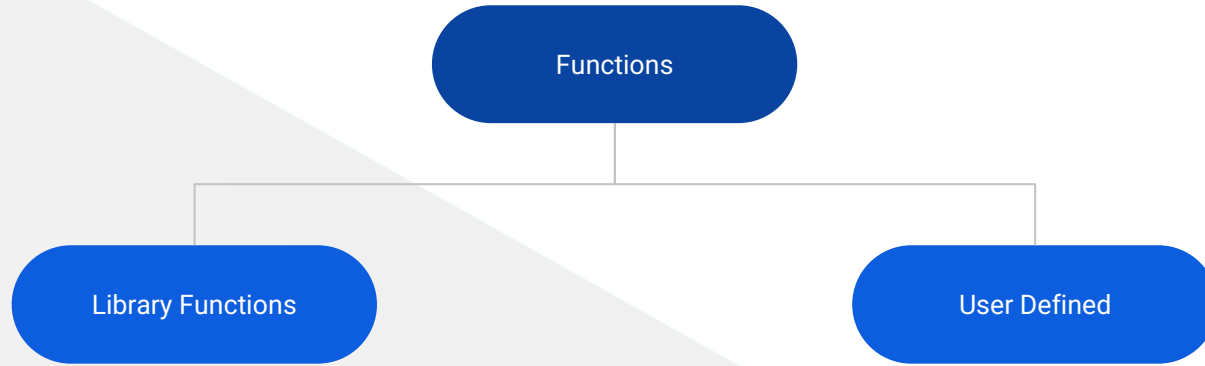
Why do you think structuring like this helps?

1. **Reuse.** Same function can be used in any program rather than writing the same code from scratch.
2. **Structured.** Code can be written in a structured way.
3. **Collboaration.** In large scale developments, different parts can be written by different coders and then they can merge to make the final software.
Think about it, clearly facebook or windows in your PC or android on your phone wasn't written by a single person.
4. **Easier, Shorter, More Efficient.**

Functions

- ⦿ Functions are a “block of code” inside a program that does a specific task.
- ⦿ You can call it anywhere inside the program, provided that it’s “defined” or “included”.
- ⦿ You may also pass some values to the function. Called function parameters.
- ⦿ You may also return a result.
- ⦿ We have been using a lot of functions already. Remember `printf()`, `scanf()`, `gets()`, `puts()`, `strlen()`, `strstr()`....?

Types of Functions



All functions that we “include” at the top of C program. They are-

- **Pre-defined.** That is someone already wrote what task they do and how they do it.
- Can be **called anywhere** inside `main()` if we include the particular library (`<stdio.h>`) at the top.

- We'll define them as per our need.
- Once defined, we can use it anywhere inside our program.

Example

Define the function

Function definition

My_function



```
int main() {  
    My_function()  
  
    My_function()  
  
    My_function() }  

```

Call it anywhere after defined

User Defined Functions

“**Define**” the function after `#include<stdio.h>` statements. Then, “**Call**” it anywhere when needed.

When defining, you need to define 4 things:

1. The name of the function, of course. You'll call it by it's name.
2. What the function takes as input. Called function “**parameters**” or “**arguments**”. (optional)
3. What the function “**returns**” as output. (optional)
4. And most importantly, what the functions does and how it does it. Called function “**body**” or “**function definition**”.

These are denoted in the code in the next slide.

Defining a function

```
1  #include<stdio.h>
2
3  float maxoftwo_func (float a, float b)
4  {
5      if (a>b)
6          return a;
7      else
8          return b;
9  }
10
11 int main()
12 {
```

Return Type (3)

Function Name (1)

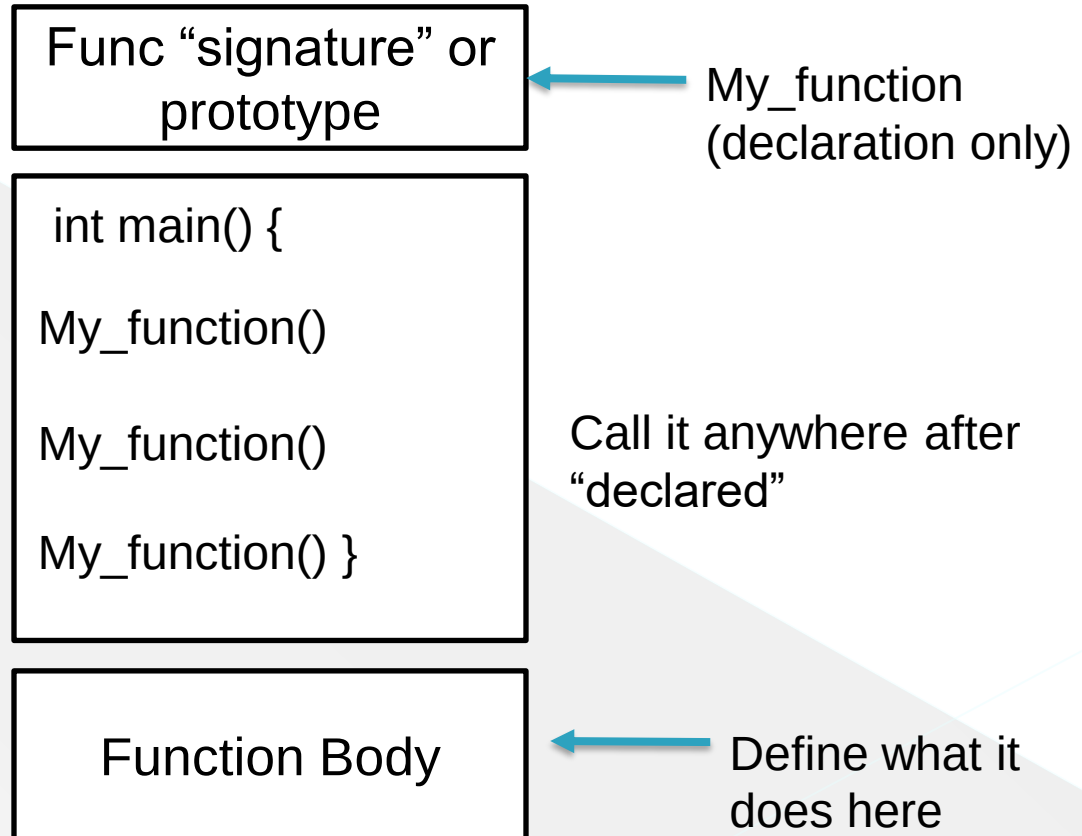
Function arguments (2)

Function body (4)

Calling a Function

```
11  int main()  
12  {  
13      float CT1, CT2, CT3;  
14      float max_of_2, max_of_3;  
15  
16      printf("\nEnter CT1 Marks: ");  
17      scanf("%f",&CT1);  
18      printf("\nEnter CT2 Marks: ");  
19      scanf("%f",&CT2);  
20      printf("\nEnter CT3 Marks: ");  
21      scanf("%f",&CT3);  
22  
23      max_of_2= maxoftwo_func (CT1,CT2);    Called here  
24      max_of_3= maxoftwo_func(max_of_2,CT3); Called here again  
25  
26      printf("\nThe maximum mark you got in your CT is: %.2f", maxoft
```

Another Declaration method for Functions



Let's see an example

a) How to define functions in this way

b) How to pass and return arrays and strings
to functions (next class)

How a function works

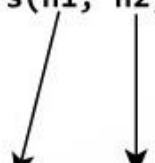
How to pass arguments to a function?

```
#include <stdio.h>

int addNumbers(int a, int b);

int main()
{
    ... ..
    sum = addNumbers(n1, n2);
    ... ..
}

int addNumbers(int a, int b)
{
    ... ..
    ... ..
}
```



The diagram shows two arrows originating from the arguments 'n1' and 'n2' in the function call 'sum = addNumbers(n1, n2);' within the main function. These arrows point down to the parameters 'a' and 'b' in the function definition 'int addNumbers(int a, int b)'. This illustrates how the values of 'n1' and 'n2' are passed to the function as 'a' and 'b'.

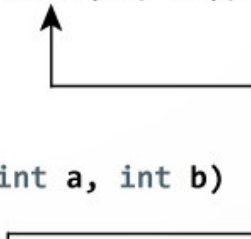
Return statement of a Function

```
#include <stdio.h>

int addNumbers(int a, int b);

int main()
{
    ... ..
    sum = addNumbers(n1, n2);
    ... ..
}

int addNumbers(int a, int b)
{
    ... ..
    return result;
}
```



The diagram shows a line from the 'return result;' statement in the 'addNumbers' function, which goes up and then left to point at the 'sum = addNumbers(n1, n2);' line in the 'main' function. This indicates that the value 'result' is being sent back to the 'sum' variable in the main function.

sum = result

Scopes of Variables inside a function

- Local variables are the one declared inside or passed to a function.
They can only be accessed inside that function only.
- Global variables are declared below `#include` statements and can be accessed by all functions of the program.
- For example, in the highest CT marks example, `a` & `b` are two local variables inside `maxoftwo_func()` and they can be accessed inside that function only.
- Similarly, `CT1`, `CT2`, `CT3`, `max_of_2`, `max_of_3` are local variables of `main()` function and can only be accessed inside `main()`.

Making your own header files with user defined functions

- ① Create a file named “something.h”. Make sure you give the extension .h and keep it in the same folder as your .c file.
- ② Use codeblocks to open and write your functions inside the something.h file.
That's it, you've created the header file.
- ③ Then simply include this header in the program you want to use it in. Write `#include “something.h”` in your target program. Notice that we use “” for user defined libraries and not `<>`.

Types of Function

⦿ A function must have a name and a body. But it may or may not have a **return value** and **input parameters**. So, based on that, they can be of 4 types-

1. No return value, no input parameters.
2. No return value, has input parameters.
3. Has return value, no input parameters,
4. Has return value, has input parameters.

A function may be written in all 4 of those ways. It's your job to pick the one best suited for your situation.

Let's try out some examples together!

Example 1

- ⦿ Let's find out in which CT a student got the better mark in between two CTs.
- ⦿ What if we had to find out the highest mark between 3 CTs instead?
- ⦿ And what if it was 4 CTs?

Functions could be helpful.

Nested function calls could be even more helpful (and memory efficient!).

Example 2

- ⦿ Let's try to find out the max number between any set of given numbers.
- ⦿ The user will tell us how many numbers to find out the max number from. And then they'll input the numbers.
- ⦿ We'll find the max among them.
- ⦿ Again, functions (and arrays) seem like the way to solve this problem.

What if we wanted to know the position of the max number as well? (Could be solved with variable scopes or with returning with pointer (but that's for next class))

Example 3

- Let's change that program to find the min number from a set of given numbers.

Example 4

- Let's see how we can use other useful library functions as well!

Example 5

- ⦿ Let's try to find out if a number is prime or not. But let's use function to do that.
- ⦿ Why functions? Could be useful! We'll find out right in the next example.

Example 6

- ⦿ Let's try to find out all the prime numbers between a range of numbers.
- ⦿ We need to check every number in a range (say from 0-100) and check if it's prime or not. If it is, let's print it.
- ⦿ We could use a nested loop. But that would mean writing the checking prime number part again! We already have that code from previous example.
- ⦿ Why not make a **header file of my own with my own function** and use that here? Sure would save some time!

Example 7

- Most numbers can be expressed as a sum of two prime numbers (usually in many combinations).
- It'll be pretty tiring to find out the combinations by hand. So, let's make our computer do it for us.
- Also, we'll write as little new code as possible by including already written functions in my own header file.

It's your turn!

Let's make your own “casino game” (jackpot arcade games) using functions.

[Click here](#) to read the rules of the game you'll be making!

Limitations

You may return anything from a function, int, float, strings (`char a[]`), arrays (`int a[]`)

But, you may return only one variable from a function. That's quite a problem.

To overcome this we use pointers and call by reference. Let's see that in next class!

Recursion

Follow the video lecture along with the solved problem in class.

You **should** also practice solving further problems with recursion, since recursion may not be an easy concept to grasp at first. Check out the problems linked below.

Links: [Here](#), [Here](#), [Here](#) and [Here](#)

Resources

1. <https://beginnersbook.com/2014/01/c-functions-examples/>
2. <https://www.w3schools.in/c-tutorial/functions/>
3. <https://www.programiz.com/c-programming/types-user-defined-functions>
4. Chapter 6, C-The Complete Reference (4th Ed), herbert Schildt

Thank You